Validating AI coding assistants under ISO 13485

A two-page guide for QA/RA leads · AI Coding Tool Validation Kit v0.2.0 · Bernardo Plaza

Why now

- **It is quality, not paperwork.** Tools that write or change product code, tests or quality records have a direct impact on quality. Validate them properly: your QA/RA lead will be enormously grateful.
- **No shadow AI.** Undeclared tools, personal accounts and unlabelled AI-assisted changes cannot be validated, controlled or defended in an audit. Declare every tool, use organisation accounts and label AI-assisted changes; the tool-discovery checklist and `checks/find_ai_footprint.py` find what is already in use.
- **The rules are in force.** The FDA QMSR has applied since 2 February 2026 and incorporates ISO 13485:2016, including clause 4.1.6: validate QMS software before initial use and, as appropriate, after changes, proportionate to risk, with records. FDA's Computer Software Assurance guidance shows how to scale assurance to risk.
- **The tools now act.** Agents edit files, run commands, open pull requests and run in CI or the vendor's cloud; on GitHub, an AI review approval can count toward merge requirements.
- **Regulators look at oversight, not at the model.** In April 2026 FDA cited a firm whose AI agents had written GMP documents that no quality unit reviewed (docs/regulatory-mapping.md, "Signals, not requirements"). FDA did not prohibit AI; it required the missing human review and approval.
- **Models and tools change constantly.** Without a change protocol a validation goes stale; the kit admits new models through a fixed suite and client versions through an automated regression run.

What to validate and what not

Validate the **intended use** (who, which tool and surface, for what, where, who approves), the **guardrails** (approval gates, required checks, permissions, sandbox, managed configuration, accounts, attribution) and the **verification of output** (human review plus CI on every AI-assisted change).

Do not try to qualify the LLM or judge whether its code is "good": review and CI do that, and they are what you validate. Exception: AI output that itself counts as a control or as evidence (an AI reviewer counted as verification); then the model is pinned and tested before every change.

The unit of validation is **tool + vendor + surface + context of use**. The binding controls sit on the code host and in CI, not in the agent's local settings.

The tests that matter

Lite runs these as 12 test records covering 14 test IDs:

1. **IQ-03** The configuration baseline is captured by script and hashed.
2. **OQ-01** Real AI-assisted PRs (at least 3 per repository, 2 repositories when available) were merged only after a qualified human approval.
3. **OQ-02** A canary PR cannot be merged without approval.
4. **OQ-03 / OQ-06** A direct push to the protected branch is blocked; an admin bypass is prevented or logged and reviewed.
5. **OQ-04** Approvals by AI, bots or CI tokens do not count, and AI identities cannot merge.
6. **OQ-05** The author, or whoever launched the agent, cannot satisfy the approval.
7. **OQ-07** Required checks block a failing test, an unapproved dependency and a secret.
8. **OQ-08 / OQ-09** Local guardrails hold (denied command, write outside the workspace, network), and users cannot override managed settings.
9. **OQ-10** A model outside the approved list is refused where the tool enforces the list, and the model used is recorded.
10. **MA-01** The fixed admission suite passes for every model in the envelope.
11. **PQ-01** In use after release for at least 4 weeks or 20 AI-assisted PRs: every merge had a qualified human approval, no bypass occurred, every model seen was inside the envelope.
12. **PQ-02** A post-merge re-review of sampled AI-assisted PRs shows that review catches defects.

Add OQ-20 (source-citation probes) when AI drafts controlled documents, OQ-30 (oracle-based verification) for autonomous QMS agents and OQ-40 (prompt injection) when agents read untrusted content. A gate never seen to refuse something has not been tested.

Start lite, go in-depth when needed

Start with **lite**, the default: per tool a plan, a test record with the tests above and a report, enough when every AI output is independently verified by human review and CI. Use **full** when any condition L1 to L9 of the lite plan applies, such as a pinned model, an agent writing to QMS systems outside the code-host gate, AI output used as evidence without review or AI-drafted controlled records without independent verification; for hard stops (L1, L2, L8, L9) the kit refuses lite. Same requirement and test IDs and commands: upgrading adds records and the full-only steps (docs/lite-vs-full.md).

Minimum record set

Record	Full	Lite
Inventory, intended use, QMS relevance, GAMP category	01	01 plan
Control objectives (REQ-01 to REQ-17; lite: a subset)	02	01 plan
AI failure modes, controls, criticality	03	01 plan
Plan and protocol, approved **before** execution	04, 05	01, 02
Configuration baseline with file hashes	06	02 (IQ-03)
Evidence exports (JSON, SHA-256, UTC timestamps)	`evidence/`	`evidence/`
Report with release decision, signed by the people themselves	07	03
Change triggers, admission records, periodic review	08	03
Work instruction, training records, tool discovery	00-shared	00-shared

Keep records for at least the device lifetime and not less than two years from release (ISO 13485 clause 4.2.5); in the EU, technical-documentation records follow MDR Article 10(8). Export code-host logs, which expire sooner.

Models change: how to stay validated

Write a **predetermined change protocol** for models into the validation plan before the first test:

1. **Initial validation**, once per tool and vendor: IQ/OQ/PQ of the guardrails, plus MA-01 on every model in the initial envelope. This fixes the suite version and its criteria.
2. **Model admission**, per new model id matching a pre-authorised pattern (never an alias the vendor can repoint): rerun MA-01. Guardrail criteria pass in every run, functional criteria in at least 2 of 3. Pass: the exact id joins the allow-list (enforced where the tool can) with an admission record. Fail: not admitted; a deviation if it was already in use.
3. **Re-baseline**: a new model family adds a targeted OQ subset; a new vendor or tool needs a new initial validation.

Envelope policy (default): any admitted model, because output is verified downstream. **Pinned** policy (AI output used as a control or as evidence): one exact model id.

When to revalidate

- New model or model version: MA-01 (envelope), or OQ subset plus MA-01 (pinned).
- Guardrail-relevant tool release, new surface or context of use, configuration change: targeted OQ subset.
- Vendor terms change (retention, training, residency): IQ-05 and a supplier review.
- Code-host rule change, incident or deviation: the gate regression tests.
- Periodic review every 12 months (6 for high-risk contexts), including tool discovery.

Next steps

1. Run `python3 kit.py init` and render your package; the wizard recommends lite or full from your answers.
2. Explore: run `checks/find_ai_footprint.py` and `checks/check_github_gates.py` on your main repositories with `--out evidence/_exploratory/`. Exploratory runs only inform the plan; IQ-03, OQ-01 and MA-01 are re-run after approval.
3. Agree the contexts of use and the model policy with engineering, then approve the plan and protocol (lite: test record).
4. Execute IQ, OQ and MA-01, release with the PQ condition, then run PQ-01 and PQ-02 and sign the addendum.
5. Schedule vendor release monitoring and the periodic review.